Codigo Multiplayer

O que faz cada parte do codigo

Projeto Cyber-Resistance-Nov25 · Godot 4.4 (C# + GDScript) · guia breve para entender a arquitetura de rede, ficheiro a ficheiro.

1. Visao geral da arquitetura

O jogo usa **duas camadas de rede ENet totalmente independentes** e um console externo do professor. Nenhuma das camadas partilha enums, autoloads ou sinais com a outra.

- **Camada 1 — Lobby / servidor central (porta 42069).** Todos os clientes (e o professor) conectam a um servidor central. Ele distribui ids de peer, lista, cria e remove salas. **Nao roda gameplay** — so coordena.
- **Camada 2 — In-game (host da sala).** Ao criar ou entrar numa sala, o cliente host abre uma segunda conexao ENet numa porta aleatoria e os demais conectam a ele. Movimento, chat, cenas e minigames trafegam aqui, fora do servidor central.
- **Console do professor.** Um cliente proprio que conecta ao servidor central, recebe as respostas das duplas e devolve nota + comentario.

Autoloads da rede (project.godot)

Autoload	Ficheiro	Papel
ProtNetworkHandler	Scripts/Multiplayer/ServerNetworkHandler.cs	Rede do servidor central (ENet)
ServerPacketHandler	Scripts/Multiplayer/ServerPacketHandler.cs	Roteador de pacotes do servidor
ClientNet	prototype/scripts/network_handler.gd	Conexao do cliente ao servidor central
ClientPacketHandler	prototype/scripts/client_packet_handler.gd	Estado do lobby no cliente
GamePacketHandler	prototype/scripts/in_game_globals/game_packet_handler.gd	Conexao in-game (host/jogador)
PlayerHostPacketHandler	prototype/scripts/in_game_globals/player_host_packet_handler.gd	Pacotes in-game -> sinais de gameplay

2. Servidor central — Scripts/Multiplayer/

`ServerMain.cs  (+ Scenes/Multiplayer/server_main.tscn)`

Tela para subir o servidor. Le os campos de IP e porta, valida o intervalo (1-65535), chama ServerNetworkHandler.StartServer() e mostra o status. GuessLocalIpv4() pre-preenche o IP local da rede (192.168.x / 10.x / 172.x).

`ServerNetworkHandler.cs  (autoload; gêmeo GDScript network_handler.gd)`

Camada de rede pura. Cria o host ENet (CreateHostBound) e, a cada frame, consome os eventos (Service()): **Connect** → atribui um id (0-255) da lista, envia PEER_ID e depois REFRESH; **Disconnect** → devolve o id ao pool; **Receive** → repassa o pacote ao roteador via

from_client_packet. Grava tambem um CSV de RTT/perda a cada 1s (enet_stats_server_*.csv, avaliacao de TCC).

`ServerPacketHandler.cs  (autoload; gêmeo server_packet_handler.gd)`
Roteador: le o byte 0 do pacote e despacha cada fluxo do lobby:

- **ROOM_REQUEST** → reserva um room_id e responde START_ROOM ao criador.
- **ROOM_INFO** → cria a RoomStorage da sala (ip/porta do host, jogadores).
- **JOIN_REQUEST** → valida sala existente e nao-cheia, adiciona o jogador, responde JOIN_ROOM ao quem entrou e avisa os demais com HAS_JOINED.
- **QUIT_ROOM_REQUEST** → remove o jogador; se quem sai e o host, fecha a sala inteira e devolve o room_id ao pool.
- **REFRESH_REQUEST** → manda a lista de salas; alem disso, um Timer de 5s empurra REFRESH periodico para quem esta no lobby (nao para quem esta em jogo).
- **MINIGAME_SUBMISSION** → guarda as respostas da sala e repassa ao professor (so aceita vinda do host da sala — evita injecao por cliente malicioso).
- **PROFESSOR_HELLO** → registra o peer como professor e lhe envia as submissoes pendentes.
- **MINIGAME_GRADE** → so aceita do professor; encaminha o pacote ao host da sala.

`RoomStorage.cs  (gêmeo storage_classes/room_storage.gd)`
Estrutura de dados de uma sala: IP e porta do host, o ENetPacketPeer do host e listas paralelas de peers, ids e nomes dos jogadores (Add/Remove Player, PlayerId, PlayerName).

`packets/  +  packets/bases/`
Todas as classes de pacote da camada 1 (ver capitulo 3). Os ficheiros .gd e .cs sao versoes equivalentes: o servidor roda a versao C# e o cliente standalone roda a GDScript, partilhando o mesmo formato binario.

3. Pacotes do lobby — Scripts/Multiplayer/packets/

Formato binario: **byte 0 = packet_type**, seguido do payload (little-endian + UTF-8). Cada pacote e uma classe com Create() (para o remetente) e CreateFromData() (para o receptor).

Bases

`packets/bases/PacketTypeClass.cs`
Classe pai de todos os pacotes: enum PACKET_TYPE, flag (reliable por omissao) e os metodos Encode(), Decode(), Send() e Broadcast().

`packets/bases/PacketCodec.cs`
Utilitarios de serializacao (u8/u16/u32/float, UTF-8, slice, resize). Little-endian igual ao do Godot — qualquer alteracao aqui quebra a compatibilidade com o cliente GDScript.

`packets/bases/RoomSummary.cs`
Resumo de uma sala (id, numero de jogadores, nomes) usado dentro do REFRESH.

Tipos de pacote (byte 0)

Tipo (valor)	De → Para	Funcao
PEER_ID (0)	servidor → cliente	Entrega o id do peer. O cliente decide o papel com my_id % 4 == 0 (host de sala).
ROOM_REQUEST (5)	cliente → servidor	Pede para criar uma sala.
START_ROOM (10)	servidor → cliente	Devolve o room_id alocado ao criador.

Tipo (valor)	De → Para	Funcao
JOIN_REQUEST (15)	cliente → servidor	Pede entrada numa sala (id, nome do jogador).
JOIN_ROOM (20)	servidor → cliente	Dados da sala para quem entrou: ip/porta do host e ids presentes.
HAS_JOINED (21)	servidor → sala	Avisa os restantes que alguem entrou (sincroniza spawns).
HAS_QUITTED (22)	servidor → sala	Avisa que alguem saiu (remove o spawn).
QUIT_ROOM_REQUEST (25)	cliente → servidor	Pede sair da sala.
QUIT_ROOM (30)	servidor → cliente	Confirma a saida ou o fecho da sala.
REFRESH (40)	servidor → cliente/prof.	Lista de salas (conjunto de RoomSummary).
REFRESH_REQUEST (50)	cliente → servidor	Pede a lista de salas nesse momento.
ROOM_INFO (60)	host → servidor	Registra a sala: ip, porta, room_id e nome do host.
MINIGAME_SUBMISSION (70)	host → servidor → professor	Respostas e tempos das 2 duplas quando ambas terminam.
PROFESSOR_HELLO (80)	professor → servidor	Registra aquele peer como professor (handshake).
MINIGAME_GRADE (90)	professor → servidor → host	Nota + comentario de uma dupla.

Cada tipo tem a sua classe em packets/ (PeerId, RoomRequestClass, StartRoomClass, JoinRequestClass, JoinRoomClass, HasJoinedPkt, HasQuittedPkt, QuitRequestClass, QuitRoomClass, RefreshClass, RefreshRequestClass, RoomInfoClass, MinigameSubmissionPkt, ProfessorHelloPkt, MinigameGradePkt), mais os ficheiros .gd equivalentes.

4. Cliente / lobby — prototype/scripts/

`network_handler.gd  (autoload ClientNet)`

Cliente ENet do lobby. start_client(ip, port) conecta ao servidor central; a cada frame consome os eventos e emite from_server_packet, on_peer_connected e on_connection_error. Grava tambem o CSV de estatisticas do lado do cliente.

`client_packet_handler.gd  (autoload ClientPacketHandler)`

Estado do lobby no cliente. Recebe os pacotes do servidor e converte-os em sinais para a UI (created_room, room_refresh, join_room, quit_room, spawn_player_signal...). Guarda my_id, a sala atual e as cenas dos jogadores. start_room() escolhe IP/porta e sobe a camada 2 como host; join_manager() conecta-se ao host da sala; trata ainda HAS_JOINED/HAS_QUITTED (spawn/despawn) e MINIGAME_GRADE (so o host a traduz em broadcast in-game).

`client_main.gd`

Tela do lobby: conectar ao servidor, refresh da lista, criar sala, entrar, nome do jogador e ajuste do layout conforme o papel (host/jogador/professor).

`multiplayer.gd + room.gd (class RoomItem)`

UI de lista de salas do protótipo: botao de nova sala/refresh e item de sala com o botao de entrar.

`main_menu.gd`
Menu principal do jogo — load_multiplayer_scene() abre a tela multiplayer e trata erros de conexao.

`player/spawn_player.gd + player/player.gd`
Spawn dos jogadores por id e o proprio jogador: movimentacao local, animacao e envio/recebimento da posicao na camada 2 (tick de rede de 0.02s), com camera apenas no jogador local.

5. Camada in-game — prototype/scripts/in_game_globals/

`game_packet_handler.gd (autoload GamePacketHandler)`
Segunda conexao ENet, **independente do lobby**. start_host(ip, porta) cria o host da sala (porta sorteada); start_player(ip, porta) conecta a ele. Roteia os receives conforme o papel (from_player_packet no host, from_host_packet no jogador), trata connect/disconnect, cleanup_connection() ao sair da sala e o log de RTT/perda em enet_stats_game_*.csv.

`player_host_packet_handler.gd (autoload PlayerHostPacketHandler)`
Tradutor dos pacotes do jogo em sinais de gameplay: movimento, chat, troca e forcamento de cena, atribuicao/resposta/progresso de minigame e nota final. Divide sinais player_* (recebidos pelo host) e host_* (recebidos pelo jogador); setup_packet_handler() liga os handlers certos conforme o papel.

`in_game_packets/ (+ base player_base.gd — InGameTypeClass)`
Pacotes desta camada: PlayerDataPacket (id, posicao, animacao), ChatTextClass (texto + time), SceneSyncPacket/SceneForcePacket (cena atual ou imposta pelo host), MinigameAssignPkt (time e papeis), MinigameAnswerPkt, MinigameProgressPkt e MinigameGradeResultPkt (nota + comentario entregue a dupla).

6. Console do professor — Scripts/Professor/

`ProfessorNetwork.cs (gêmeo professor_network.gd)`
Cliente ENet proprio (sem autoload). Conecta ao servidor central, envia PROFESSOR_HELLO no handshake (so entao o servidor aceita notas e repassa submissoes), pede a lista de salas, envia MINIGAME_GRADE e converte os pacotes recebidos em eventos C# para a UI.

`ProfessorMain.cs (gêmeo professor_main.gd)`
Tela do professor: conectar, listar/filtrar salas, ver as submissoes de cada dupla (perguntas do JSON, respostas e tempos) e devolver nota + comentario, que volta ao host via MINIGAME_GRADE.

7. Fluxo completo, resumido

1. O aluno abre o lobby → ClientNet conecta ao servidor central e recebe PEER_ID + REFRESH (lista de salas).
2. **Criar sala:** ROOM_REQUEST → START_ROOM (room_id) → o cliente sobe a camada 2 numa porta aleatoria → ROOM_INFO → o servidor publica a sala.
3. **Entrar na sala:** JOIN_REQUEST → JOIN_ROOM (ip/porta do host) para quem entrou + HAS_JOINED para os demais → todos ligam-se a camada 2.
4. **Jogar:** movimento, chat, cenas e minigames trafegam na camada 2, com o host da sala como autoridade.

5. **Terminar o minigame:** quando as 2 duplas concluem, o host envia MINIGAME_SUBMISSION → o servidor guarda e encaminha ao professor.
6. **Corrigir:** o professor envia MINIGAME_GRADE → o servidor encaminha ao host → o host faz broadcast do MinigameGradeResultPkt para a dupla.
7. **Sair:** QUIT_ROOM_REQUEST → QUIT_ROOM/HAS_QUITTED; se o host sai, a sala fecha e o room_id volta ao pool.